Influence of the number of clients in a federated machine learning model

Research Project

Influence of the number of clients in a federated machine learning model

Marco Antonio Barrera Salas

December 8, 2025

submitted to the Institute for Data Engineering
at the Hamburg University of Technology.

Supervisors:

Pierre-Alexandre Murena
Institute for Data Engineering
TU Hamburg

TUHH

Research group on Human-Centric Machine Learning

Hamburg University of Technology
Institute for Data Engineering

Am Schwarzenberg-Campus 1
21073 Hamburg
Germany

Marco Antonio Barrera Salas
643867
Influence of the number of clients in a federated machine learning model
December 8, 2025

To innovation and collaboration in AI research.

Contents

Abstract **xiii**

1 Introduction **1**
1.1 Motivation 1
1.2 Background 2
1.3 Objectives and Research Questions 2

2 State of the Art **5**
2.1 IoT Industry and Machine Learning 5
2.2 Overview of Federated Learning 9
2.3 Related Work 12

3 Materials and Methods **17**
3.1 Experimental Framework 17
3.2 Dataset and Partitioning Strategies 22
3.3 Experiment Configuration 25

4 Results and Discussion **27**
4.1 Effect of Number of Clients 27
4.2 Impact of Random Seeds 28
4.3 Influence of Partial Client Participation 29
4.4 Non-IID Data Effects 30
4.5 Main findings in this work 33

5 Conclusion **35**
5.1 Summary of Findings 35
5.2 Implications for IoT and Real-World Applications 36
5.3 Limitations 36

6 Outlook **37**
6.1 Future Work 37
6.2 Potential Extensions 37

Appendix **45**

List of Figures

2.1 Number of connected IoT devices worldwide between 2015 and 2025[ACMF21]. 6
2.2 IoT-edge computing architecture. 7
2.3 Security system with Iot and ML. 8
2.4 Generalized FL process. 10
2.5 Timeline of key FL developments from 2016 to 2025, including FedAvg, FedProx, and other extensions. 11
2.6 Clients and server communication flow and topology. 12
2.7 Data distribution in different clients. 14
2.8 Impact of the number of clients on test accuracy. 15
2.9 Impact of the number of clients on exchange time. 16
2.10 Impact of the data Heterogeneity. 16

3.1 Federated learning workflow used in all experiments. 18
3.2 Overview of the classes in the MNIST fashion dataset 19
3.3 Dirichlet Partitioner Alpha 10.0 23
3.4 Dirichlet Partitioner Alpha 0.01 23
3.5 Partitioners 24
3.6 Shard partitioner random combination example 25

4.1 Accuracy and loss metrics through rounds 28
4.2 Overview of the model accuracy with multiple seeds 28
4.3 Partial Client Participation analysis 30
4.4 Overview of different data distributions 31
4.5 Influence of client distribution throughout rounds 31
4.6 Boxplot view of the overall accuracy with shard distribution . 32
4.7 Convergence error lines with shard distributions 33

1 The loss behavior for different client participation 45
2 The accuracy behavior for different client participation 45
3 The loss behavior for different data distributions 45
4 Histogram view for different data distributions 46
5 Data behavior for factor 0.1 in the partitioner 46
6 Data behavior for factor 0.1 in the partitioner through rounds 46
7 Heatmap for the data distribution with Dirichlet partitioner of alpha 0.01 47
8 Percentage barplot for the data distribution with Dirichlet partitioner of alpha 0.01 47

9 Heatmap for the data distribution with Dirichlet partitioner of alpha 10.0 47
10 Percentage barplot for the data distribution with Dirichlet partitioner of alpha 10.0 48

List of Tables

2.1 Main advantages of IoT integration 7
2.2 Challenges of ML deployment in IoT systems. 9
2.3 Advantages and disadvantages of Federated Learning compared to centralized machine learning. 13
2.4 Comparison of selected federated-learning frameworks. . . . 14
2.5 Comparison of Federated Learning approaches for IoT intrusion detection. 15

3.1 Overview of client and server responsibilities in Flower. 20
3.2 Additional functionality provided by the `CustomFedAvg` strategy. 21
3.3 Client dataset sizes under different Dirichlet α values (10 clients). Lower α yields higher imbalance. 22
3.4 Federated learning experiment configurations. 25

4.1 Summarized keypoints. 29

Abstract

In this work, the behavior and performance of Federated Machine Learning (FML) systems are investigated under different data distribution and participation settings. Federated Learning is an emerging paradigm that enables multiple clients or edge devices to collaboratively train a global model without sharing their raw data, thus preserving privacy and reducing communication costs. This decentralized approach is particularly relevant for Internet of Things (IoT) ecosystems, where large amounts of data are generated by geographically distributed and resource-constrained devices. By allowing on-device training and model aggregation, FML offers a scalable and privacy-preserving alternative to traditional centralized learning.

The project focuses on three main aspects that influence the overall performance of an FML system: (1) the number of participating clients, (2) the degree of data heterogeneity across clients (IID vs. non-IID distributions), and (3) the fraction of clients participating in each training round. A series of controlled experiments were conducted using the Flower framework, varying the client count, random seeds, Dirichlet distribution parameters, and participation fractions to analyze these effects.

Results show that increasing the number of clients generally improves model stability across random seeds, though overall accuracy remains similar due to reduced data per client. Furthermore, strongly non-IID data distributions were found to significantly reduce model accuracy and slow convergence. This effect mirrors real-world IoT scenarios, where devices differ in sensing modalities, usage patterns, or hardware-software configurations, leading to unbalanced and biased local datasets. Such heterogeneity can degrade global performance unless adaptive aggregation, clustering, or personalization strategies are employed. Lastly, varying the client participation fraction (from 0.2 to 1.0) had minimal impact on convergence speed or final accuracy, suggesting that federated learning remains robust even under intermittent connectivity, which is a common challenge in large-scale IoT deployments.

Overall, this work highlights key trade-offs between scalability, data heterogeneity, and client participation in Federated Learning. The findings provide practical insights for developing resilient and efficient IoT-oriented federated systems capable of maintaining reliable performance despite uneven data distributions and unreliable communication conditions.

1

Introduction

This chapter provides an overview of the context, motivation, and objectives of this work. It introduces the relevance of Federated Machine Learning (FML) within the Internet of Things (IoT) ecosystem, outlines the problem addressed, and presents the guiding research questions explored throughout this study.

1.1 Motivation

In recent years, the number of Internet of Things (IoT) devices has grown exponentially, leading to a massive increase in data generated by sensors, edge devices, and connected systems. Traditionally, machine learning models rely on centralized data collection, where information from all devices is transmitted to a central server for processing and training. However, this approach faces several challenges in modern distributed environments. Data privacy regulations (such as GDPR), limited bandwidth, high communication costs, and device heterogeneity make centralized learning inefficient and often impractical.

Federated Machine Learning (FML) has emerged as a promising paradigm to overcome these challenges. Instead of sending raw data to a central server, each device (or client) trains a local model using its own data, and only the learned parameters are shared for aggregation. This approach preserves data privacy, reduces transmission overhead, and enables learning directly at the edge of the network. In the context of IoT, this paradigm aligns naturally with the distributed nature of connected devices and opens new possibilities for scalable, privacy-preserving intelligence across heterogeneous systems.

1.2 Background

Federated Learning (FL) was first introduced by Google in 2016 as a distributed learning framework designed to train models collaboratively without centralizing data, as the author [MMR+17] mentions in his work. It consists of three key steps: local training at each client, aggregation of model parameters at the central server, and global model updates distributed back to the clients. Over several communication rounds, the global model converges toward a solution that reflects the knowledge of all clients.

When applied to IoT systems, Federated Learning offers a practical and privacy-friendly solution for deploying AI applications such as predictive maintenance, anomaly detection, or user personalization, as seen in [LNJK+20, LSTS20]. Nevertheless, IoT environments are highly dynamic—devices vary in computational capacity, communication stability, and data characteristics. This heterogeneity introduces major challenges such as non-IID (non-independent and identically distributed) data, partial client participation, and synchronization issues, according to [KMA+21]. Addressing these challenges is essential for achieving robust and efficient learning across diverse and unreliable IoT networks.

1.3 Objectives and Research Questions

The main objective of this work is to analyze how different configuration parameters influence the performance and convergence behavior of Federated Machine Learning systems in IoT-oriented scenarios. Specifically, the study focuses on three aspects:

- **Number of clients:** Understanding how the number of participating clients affects global accuracy, convergence speed, and result stability across random seeds.
- **Data distribution:** Investigating the impact of IID versus non-IID data distributions on model performance, reflecting the heterogeneity typically found in IoT environments.
- **Client participation:** Assessing how varying the fraction of participating clients in each round influences global training dynamics and overall robustness.

These objectives lead to the following research questions:

1. How does the number of participating clients influence the global model's performance and stability?
2. How does data heterogeneity (IID vs. non-IID) affect convergence and accuracy in federated training?
3. To what extent does partial client participation impact the overall performance of the federated model?

Studying these questions is essential for understanding how Federated Machine Learning behaves under realistic IoT conditions, where devices may have limited resources, unreliable connectivity, and non-uniform data distributions. The insights derived from this work aim to support the design of robust and efficient federated learning systems capable of maintaining reliable performance in real-world distributed environments.

2

State of the Art

2.1 IoT Industry and Machine Learning

The Internet of Things (IoT) refers to the interconnection of physical devices capable of collecting, transmitting, and processing data through networked communication systems. It represents a major technological shift that enables real-time monitoring, control, and optimization across diverse domains such as smart homes, healthcare, manufacturing, and transportation, as discussed in [AIM10]. The scale of this ecosystem has expanded exponentially: billions of sensors and embedded devices now generate continuous data streams that require intelligent analysis, as seen in [ZBC+14].

Machine Learning (ML) plays a pivotal role in transforming this raw sensor data into actionable insights. By detecting patterns, predicting failures, or optimizing energy consumption, ML empowers IoT systems to operate autonomously and efficiently. Traditionally, ML models have relied on centralized data collection in the cloud, which raises issues of latency, privacy, and communication overhead. To mitigate these challenges, recent years have seen a shift toward *edge intelligence*, which brings computation closer to where data is generated, as mentioned in [LZ18]. Some of these real applications can be predictive maintenance in industrial equipment based on vibration or temperature data, personalized healthcare monitoring using wearable sensors, and traffic optimization and adaptive lighting in smart cities.

According to the author in the work [ACMF21], the number of IoT devices had an exponential growth during the last years, as it can be seen below in figure 2.1.

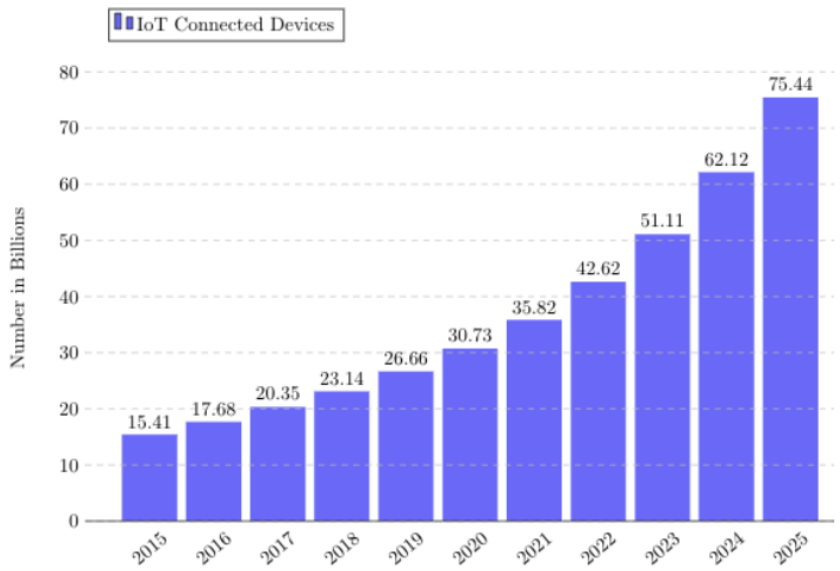


Figure 2.1: Number of connected IoT devices worldwide between 2015 and 2025[ACMF21].

2.1.1 Evolution of the IoT Ecosystem

The concept of the Internet of Things (IoT) has evolved from early ideas of simple connected sensors to a vast, intelligent ecosystem of devices, systems, and services. Initially, IoT was driven by the integration of Radio Frequency Identification (RFID) and Wireless Sensor Networks (WSN), enabling basic device connectivity and data acquisition, as shown in [GBMP13]. Over time, technological advances in embedded systems, cloud computing, and network infrastructure have transformed IoT from a vision into an operational reality.

The first generation of IoT systems relied heavily on centralized cloud architectures, where data from distributed sensors was transmitted to remote servers for storage and processing. Although this architecture provided scalability and computational resources, it introduced challenges such as high latency, bandwidth limitations, and privacy concerns. To address these limitations, the paradigm gradually shifted toward *edge computing* and *fog computing*, where data processing occurs closer to the source, mentioned in [SCZ+16].

This transition has enabled real-time decision-making and reduced dependency on centralized infrastructures. In modern implementations, IoT systems often combine cloud, edge, and on-device intelligence, creating a hierarchical computing model optimized for performance and efficiency. The convergence of IoT with Artificial Intelligence (AI) and Machine Learning (ML) has further accelerated innovation which allows connected devices to not only sense and communicate but also learn, adapt, and act autonomously [ZCL+19].

Overall, the evolution of IoT reflects a shift from passive data collection to

Key technological enables	Description
Connectivity	The proliferation of 5G, LPWAN, and mesh networking technologies enabling large-scale device interconnectivity.
Computation	Advances in microprocessors and embedded AI chips supporting local inference.
Data management	Cloud edge integration for efficient storage, analysis, and lifecycle management of IoT data.

Table 2.1: Main advantages of IoT integration

active, intelligent systems that continuously learn from their environments and improve operational outcomes. An example of this scenario is the work of the author [ZYZM23] in figure 2.2, where they implemented an enhanced version of an IoT system by applying edge computing, which is not covered in this work. However, this leads to the next subsection, where IoT applications utilized machine learning techniques.

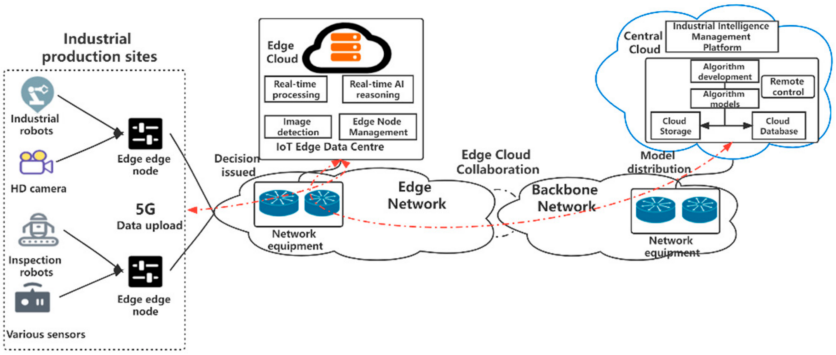


Figure 2.2: IoT-edge computing architecture.

2.1.2 Machine Learning in IoT Applications

The integration of Machine Learning (ML) into IoT systems has nowadays revolutionized how connected devices perceive, interpret, and respond to environmental data. ML algorithms can automatically identify complex patterns and make predictions that enhance system reliability, efficiency, and user experience. For instance, supervised models are used for classification and regression tasks such as fault detection or energy forecasting, while unsupervised approaches help uncover hidden patterns in sensor data for anomaly detection or clustering, as mentioned in the work of the author [ZWCW21].

On the other hand, ML-driven IoT systems in industrial settings enable predictive maintenance by analyzing real-time vibration or temperature data

to anticipate equipment failures before they occur. In smart cities, ML optimizes traffic flow, manages energy grids, and supports adaptive lighting systems. In healthcare, wearable IoT devices coupled with ML algorithms monitor patient vital signs, detect irregularities, and provide early alerts for medical intervention, as described by [MMG21].

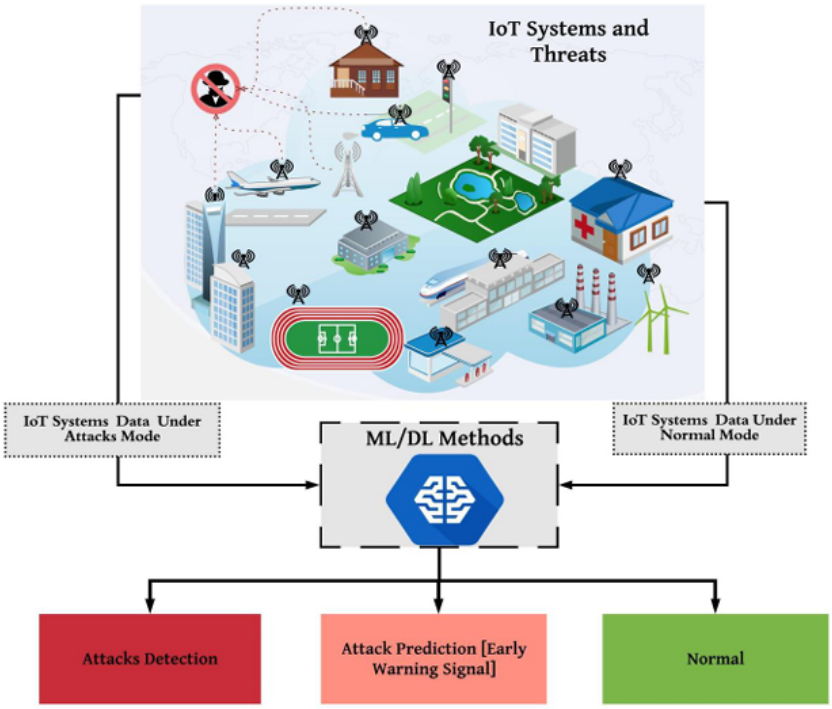


Figure 2.3: Security system with Iot and ML.

Additionally, a real example of an IoT application with a machine learning model can be seen in figure 2.3, where the main goal is to utilize an Iot system to detect threats by using sensors and machine learning methods, as described in the paper of [AMA+20]. Furthermore, it is also mentioned that machine learning can deep learning (DL) methods can be considered as powerful data exploration methods to learn about "normal" and "abnormal" data behavior according to how IoT components and devices interact with each other within an IoT system.However, the successful deployment of ML in IoT comes with several challenges, as it can be seen in table 2.1.

These limitations have motivated the emergence of *Federated Learning* (FL), a decentralized training paradigm that allows multiple IoT devices to collaboratively train shared ML models without exchanging raw data [MMR+17]. FL has become a cornerstone for privacy-preserving AI in IoT ecosystems, as it aligns with the natural distribution of data while reducing communication overhead and preserving confidentiality. However, this will be further discussed in the next section of this work.

Challenge	Description
Data heterogeneity	IoT data can be noisy, incomplete, or inconsistent across devices and environments.
Resource constraint	Edge devices often have limited computational power and storage capacity, restricting the complexity of models that can be trained or deployed locally.
Privacy and security	Centralized data aggregation raises risks of information leakage, regulatory non-compliance, and cyber-attacks.

Table 2.2: Challenges of ML deployment in IoT systems.

In summary, Machine Learning empowers IoT systems to move from reactive monitoring to proactive, adaptive, and autonomous decision-making, forming the foundation of intelligent, connected environments.

2.2 Overview of Federated Learning

As mentioned before, this section of the work provides an overview of federated learning and its key advantages. Addtionally, some historical background and related surveys are addressed, as they can be considered as remarkable and have a relation with the experimental part of this work.

2.2.1 Background and Evolution

Federated Learning (FL) was first introduced by Google in 2016 as a distributed training paradigm that allows multiple clients (e.g., mobile devices or edge nodes) to collaboratively learn a shared model without exchanging raw data, shown in [MMR+17]. The canonical algorithm, known as Federated Averaging (FedAvg), involves the steps shown in figure 2.4.

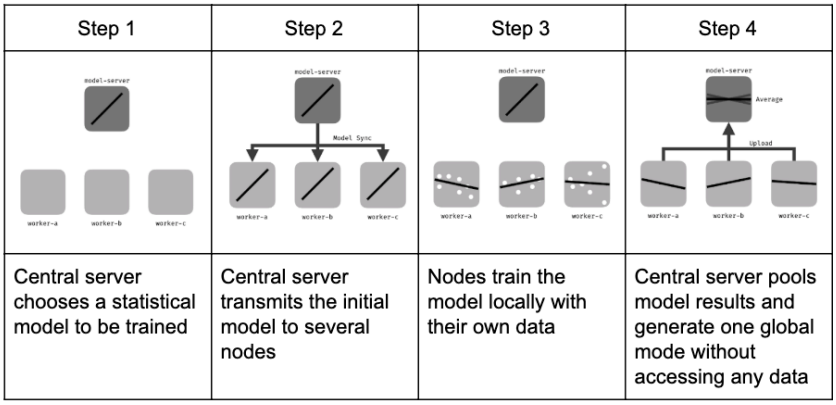


Figure 2.4: Generalized FL process.

Over multiple communication rounds, the global model converges to a solution that reflects the collective knowledge of all participants. Federated Learning can be categorized into horizontal, vertical, and transfer FL, depending on how data and features are distributed across clients, which is also discussed in the work of [KMA+21].

The timeline of Federated Learning evolution (see Fig. 2.5) illustrates the progressive development of the paradigm from early distributed and privacy-preserving learning concepts to its formal introduction in 2016 [YLCT19]. Subsequent years brought major advances, including mobile-edge deployment, cross-device and cross-silo architectures, and algorithmic strategies to handle non-IID and heterogeneous client data. More recent stages focus on domain specific applications, such as healthcare, IoT, and on-device learning, which reflects a broader shift from centralized training to collaborative, privacy-aware model development.

In the context of this work, the timeline situates our work within the modern phase of large-scale federated systems under realistic constraints. Our experiments on varying client counts and random seeds align with current research directions that emphasize system level robustness, communication efficiency, and scalability in real world federated learning deployments.

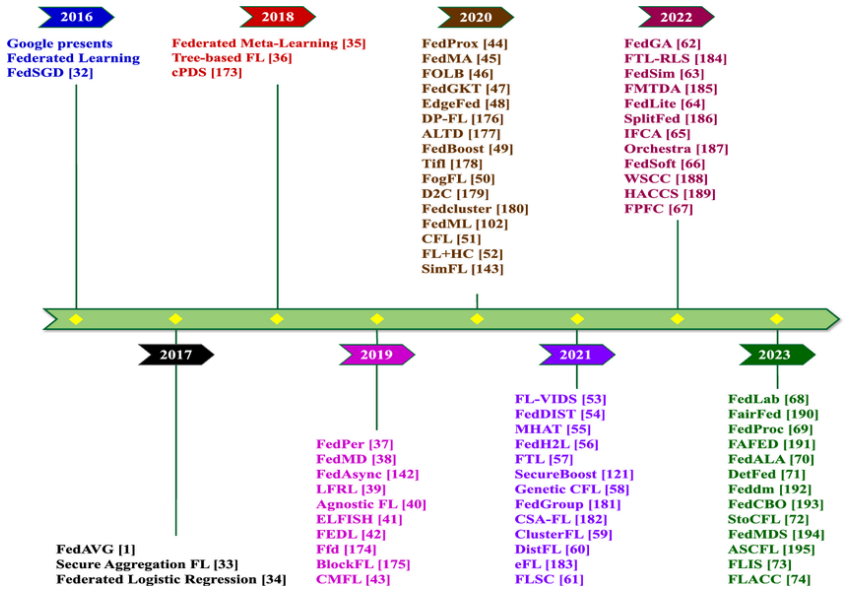


Figure 2.5: Timeline of key FL developments from 2016 to 2025, including FedAvg, FedProx, and other extensions.

2.2.2 Real Implementations and Frameworks

Since its introduction, Federated Learning has evolved from a conceptual framework to a production level technology. One of the earliest real world applications was Google's *Gboard*, where the FL framework was used to improve next-word prediction without centralizing user data, as this author mentions [BEG+19].

In healthcare, FL has enabled collaborative model training across hospitals while maintaining patient confidentiality, shown in [RHL+20]. In the IoT domain, it allows distributed devices—such as sensors or edge gateways—to jointly train predictive models, improving scalability and privacy at the same time.

Several open-source frameworks have accelerated this adoption. Notably, *Flower* provides a flexible research platform for prototyping FL systems in Python [BTM+20], while *TensorFlow Federated* and *NVIDIA FLARE* offer more production-oriented environments. However, it is important to understand that the main topology of a federated learning model consist of the same structure, as it is shown in (Fig. 2.6) and discussed by [LTC23].

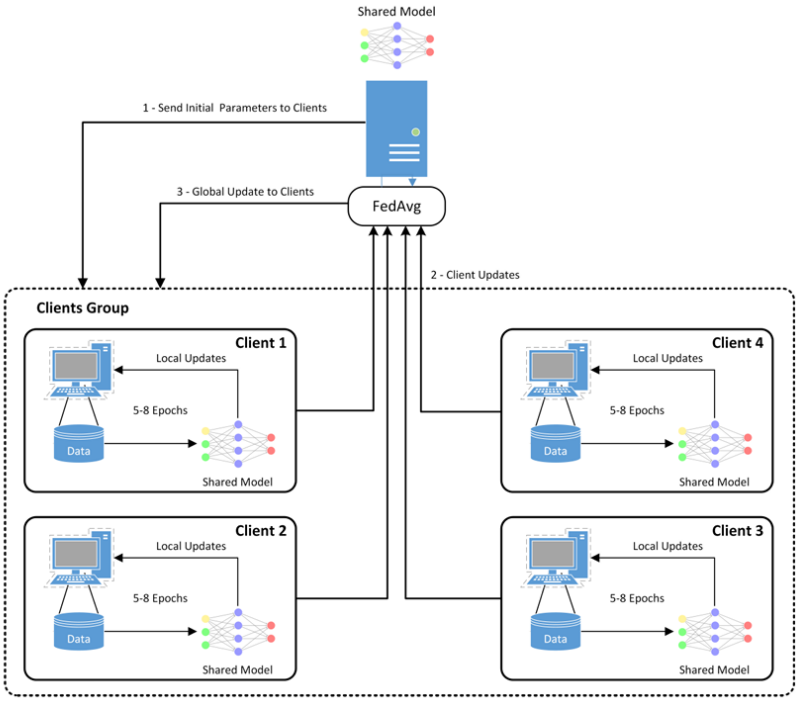


Figure 2.6: Clients and server communication flow and topology.

2.2.3 Advantages and Disadvantages

Federated Learning provides several advantages and disadvantages over traditional centralized learning, as described in (Table. 2.3)

2.3 Related Work

Several studies have extended Federated Learning to meet the constraints of IoT and edge computing. A comprehensive survey by [LNJK+20] categorizes FL techniques into client selection, model optimization, and communication reduction strategies. Other research focuses on improving convergence and robustness under heterogeneous conditions, such as *Clustered Federated Learning* [SWMS20], which groups clients based on data similarity to enhance model personalization.

Recent frameworks and methodologies emphasize adaptivity—balancing performance, energy efficiency, and privacy. These advancements are key for deploying FL in large-scale, real-world IoT environments where computational resources and connectivity vary significantly. It can be seen in (Table. 2.4), created with the information from the author in [Bra23], that nowadays multiple fameworks are utilized in a worldwide level, such as

Advantages/Disadvantages	Description
Privacy preservation	Sensitive data remains local on user devices, minimizing the risk of data exposure and improving user trust.
Regulatory compliance	Enables compliance with data-protection regulations such as GDPR and HIPAA, as raw data never leaves its source.
Scalability	Supports distributed training across large, heterogeneous networks of devices.
Non-IID data	Local datasets differ in distribution and size, leading to biased model updates and reduced global performance.
Communication bottlenecks	Frequent model synchronization between clients and server can introduce latency and network overhead.
Partial participation	Some clients may be offline or unavailable during training rounds, slowing or destabilizing convergence.
Security risks	Despite data decentralization, FL remains vulnerable to poisoning and inference attacks [LSTS20, YLCT19].

Table 2.3: Advantages and disadvantages of Federated Learning compared to centralized machine learning.

NVFlare, FATE, Flower and PySyft. This will also be discussed in the next section, but it is important to remark that this work utilizes Flower, as it is a friendly-beginner framework and allowed this work to perform the simulations required for the expected results.

The author in the work of [LTC23] compiled multiple surveys and compared the dataset and frameworks used in each one of them, which is an useful overview of what other authors are currently doing. This information can be seen in (Table. 2.5).

Additionally, the author in [WNL+23] performed several extensive experiments on a large network of real IoT and edge devices to present real-world federated learning characteristics, including learning performance, communication and computation costs in heteregeneous scenarios, which can be identified as the most challenging issue in FL. The author is utilizing a big number of devices such as Raspberry Pi and NVIDIA jetson devices. where each device contains a different amount of data quantity and quality. An overview of this can be seen in the (fig. 2.7).

Framework	Key Characteristics
NVFlare	Business-ready, framework-agnostic; supports tree-based models; strong industry backing. / Steeper learning-curve to get started.
FATE	Early commercial FL framework; includes many modules (ML, orchestration, Spark/Kubernetes support). / Less flexible for custom modules; complex usability.
Flower	Flexible, easy to use, research oriented; framework-agnostic and extendable. / Fewer built-in privacy/orchestration features relative to enterprise frameworks.
PySyft	Strong in remote data science and differential-privacy tooling; large community. / Under development; documentation and model-support limited.

Table 2.4: Comparison of selected federated-learning frameworks.

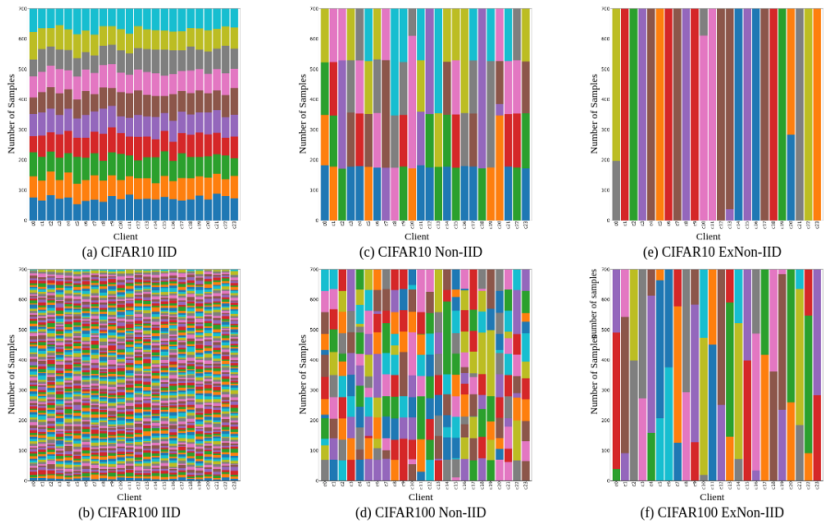


Figure 2.7: Data distribution in different clients.

One of the most important insights of this work, is that he also was interest in the influence of the number of clients, where he concludes that more participating clients do not guarantee better accuracy but can lead to large congestion in communication and increase the update exchange time. This can also be seen in (fig. 2.8) and (fig. 2.9).

Cite	Dataset	Model	Agg Func.	Clients	FL Library
[CSG+22]	NF-BoT-IoT	LSTM and DNN	FedAvg	-	-
[SLMP23]	Proprietary	LSTM	-	4	Flower
[ZYSX20]	MODBUS	GRU and Random Forest	FIAverage	-	Pysyft
[ZLLL22]	CICIDS2017	Adaboost and RF	Weighed Voting FedAvg	5	-
[MKP+22]	LANDER IoT	Auto-Encoder	-	9	-
[ZHM+21]	DS20S Traffic	MLP Stacking Ensemble	FedStacking	4	-
[ANT17]	MNIST	MLP	FogFL	6	-
[CH22]	SEA Dataset	LSTM	FedAvg	4	Tensorflow
[SMD21]	TonIOT	Logistic Regression	FedAvg	10	IBMFL
[CLL+20]	WSN-DS	GRU-SVM	FedAvg	50	Pysyft

Table 2.5: Comparison of Federated Learning approaches for IoT intrusion detection.

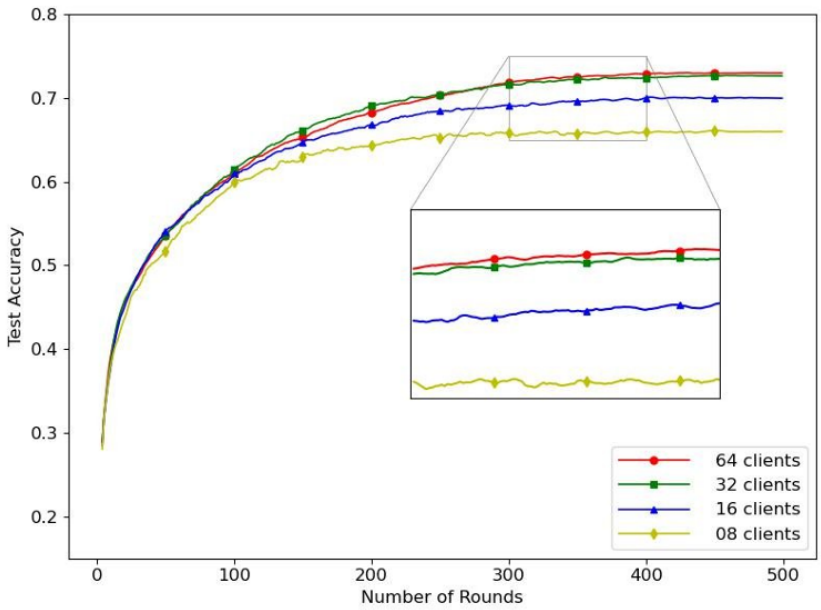


Figure 2.8: Impact of the number of clients on test accuracy.

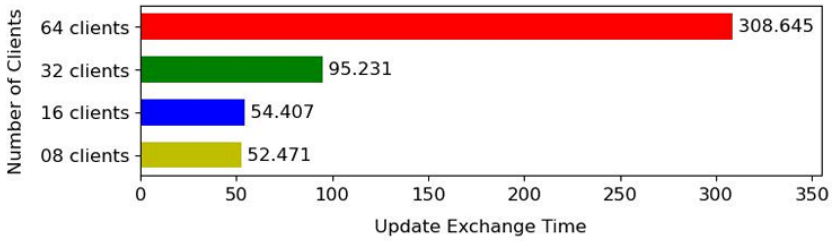
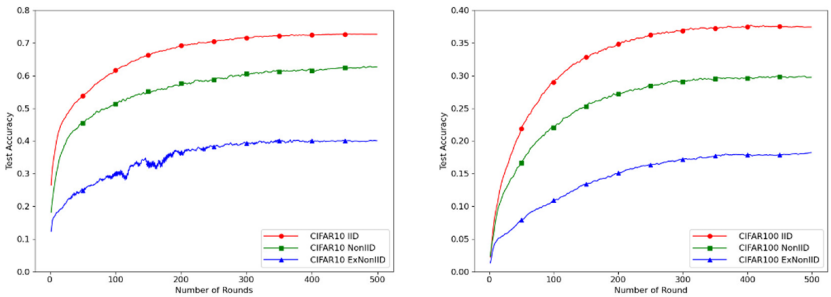


Figure 2.9: Impact of the number of clients on exchange time.

Furthermore, other of the most important insights that this work provides is an overview of how data heterogeneity affects the test accuracy, which is another of the research questions in this work and will be discussed in section 4. However, the results of this scenario can be seen in (fig. 2.10), where it can be seen that the Non-IID scenarios degrade the accuracy on test sets significantly compared to IID. This can be mainly caused by the different hardware capacities between all of the devices, as some of them have different computationl constraints, which it is related to how much data they can handle or how fast they can perform operations and then do the exchange with the server, where the bandwidth plays a big role in a federated machine learning system.



(a) CIFAR10 (b) CIFAR100

Figure 2.10: Impact of the data Heterogeneity.

Summary: This chapter provided an overview of the technological context in which Federated Learning has emerged. From the expansion of IoT ecosystems to the need for decentralized intelligence, FL offers a promising paradigm for distributed model training under privacy constraints. The following chapter will build on this foundation to describe the methodology and experimental setup of the proposed study.

3

Materials and Methods

This chapter provides an overview of not only the utilized tools, but also the programming framework, the chosen dataset,and the general configuration of the federated learning system.

3.1 Experimental Framework

3.1.1 Flower Library

As mentioned in the previous chapter, nowadays there exists multiple frameworks that are specialized for federated machine learning. However, it was decided that for this work, the most suitable option would be to utilized Flower, as it enables to create a simulation of large fedetated networks on a single machine while maintaining a realistic client-server separation. Additionally, it is considered as a friendly user and ready to use framework, as it can also be seen in the work of the author [BTM+20]. As it was also shown previously, the current federated learning flow can be seen in figure 3.1, which shows what is basically document in every round.

3.1.2 Dataset

The data utilized in this work is the Fashion-MNIST dataset [XRV17], a modern drop-in replacement for MNIST designed to provide a more challenging visual classification benchmark. The dataset consists of 60,000 training and 10,000 test images of fashion items across 10 classes (e.g., T-shirt/top, trouser, pullover, coat, sneaker).

All inputs are grayscale 28×28 images, and each label belongs to one of the 10 garment categories. Fashion-MNIST retains the structural simplicity of

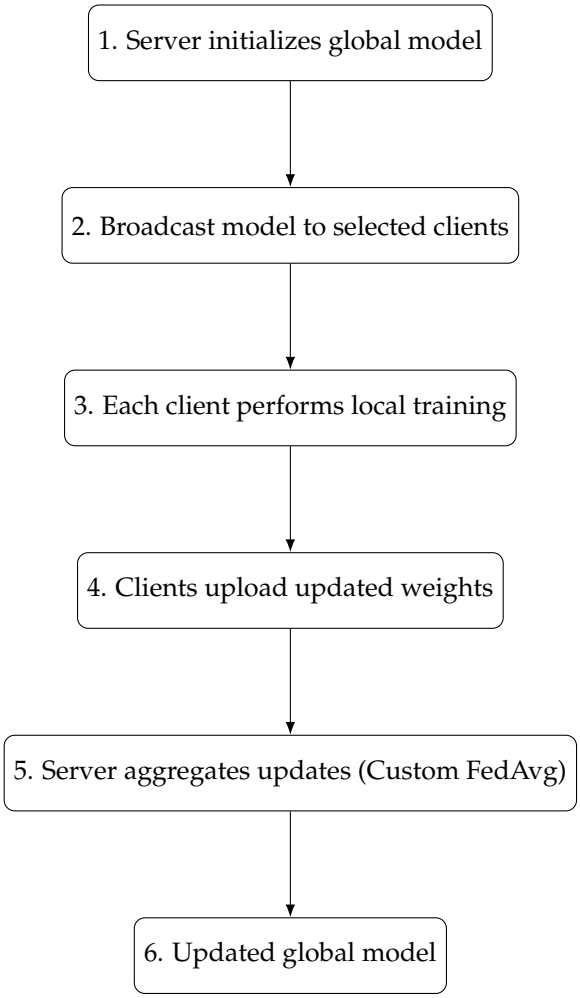


Figure 3.1: Federated learning workflow used in all experiments.

MNIST while introducing higher visual variability, making it a more realistic proxy for IoT sensor-image workloads.

This dataset is widely used in federated learning research due to its balanced classes, low storage footprint, and compatibility with heterogeneous training environments, as it can be seen in figure 3.2.

Figure 3.2: Overview of the classes in the MNIST fashion dataset

3.1.3 Convolutional Neural Network

The federated learning experiments employed a compact convolutional neural network (CNN) designed for the FashionMNIST image classification task. The architecture follows a classical convolution–pooling–fully connected design, offering a balance between computational efficiency and representational capacity. Its lightweight nature makes it suitable for deployment on resource-limited IoT and edge devices, aligning with the goals of federated learning experiments on constrained hardware.

The model processes grayscale images of size 28×28 and consists of two convolutional blocks followed by three fully connected layers. The first convolutional layer applies 6 filters of size 5×5, producing feature maps that are subsequently downsampled by a 2×2 max-pooling operation. A second convolutional layer with 16 filters of size 5×5 further extracts spatial features, followed by another max-pooling operation. The resulting activation maps are flattened into a 256-dimensional vector and passed through two hidden fully connected layers of sizes 120 and 84. The final classification layer outputs 10 logits corresponding to the FashionMNIST classes.

The forward pass can be summarized as:

$$\begin{aligned}\text{Input } (1 \times 28 \times 28) &\rightarrow \text{Conv1 } (6 \times 24 \times 24)\\ &\rightarrow \text{MaxPool } (6 \times 12 \times 12)\\ &\rightarrow \text{Conv2 } (16 \times 8 \times 8)\\ &\rightarrow \text{MaxPool } (16 \times 4 \times 4)\\ &\rightarrow \text{Flatten } (256)\\ &\rightarrow \text{FC1 } (120)\\ &\rightarrow \text{FC2 } (84)\\ &\rightarrow \text{FC3 } (10).\end{aligned}$$

The model is trained using the Adam optimizer with cross-entropy loss. This choice supports stable convergence across heterogeneous client datasets and varying partitioning schemes. The relatively low parameter count also reduces communication overhead during federated rounds.

3.1.4 Client and Server Design

It is important to remark that the client and the server in a federated machine learning system have different responsabilities, which are shown in the table 3.1. For this work, only one server will be taken into account, while the number of clients will vary depending on the experiment.

Table 3.1: Overview of client and server responsibilities in Flower.

Component	**Responsibilities**
Client	Training the local model, computing metrics, managing private data partition, and returning updated model parameters.
Server	Orchestrating federated rounds, sampling clients, aggregating updates using the custom FedAvg, and broadcasting new global models.

3.1.5 Aggregation Strategy

A modified version of the Federated Averaging (FedAvg) algorithm was utilized [MMR+17], used as a custom strategy named `CustomFedAvg`. While the core averaging rule of FedAvg remained unchanged, this extended version introduced additional functionality relevant for experiment tracking, reproducibility, and large-scale IoT-oriented FL simulations.

The standard FedAvg aggregation rule computes the global model as a

weighted average of the local parameters w_k collected from K clients:

$$w_{\text{global}} = \frac{1}{N} \sum_{k=1}^{K} n_k \, w_k,$$

where n_k denotes the number of samples held by client k. This makes FedAvg robust to heterogeneous data sizes and variable client participation—properties that are crucial in IoT scenarios where devices differ significantly in activity patterns, computational capacity, and dataset availability.

To better support monitoring and analysis during our experiments, the default FedAvg behaviour was extended by integrating automatic checkpointing, metric logging, and persistent evaluation tracking. These enhancements are summarized in Table 3.2.

Table 3.2: Additional functionality provided by the `CustomFedAvg` strategy.

Feature	**Description**
Global Model Checkpointing	After each training round, the aggregated model parameters are converted into a PyTorch state dictionary and stored as a checkpoint (e.g., `global_model_round_t`). This enables reproducibility and post-hoc analysis of round-to-round model evolution.
Persistent Evaluation Metrics	Evaluation metrics (loss and accuracy) produced after each round are collected and stored in a structured JSON file. This creates an experiment log independent of external tools.
Weights & Biases Logging	Metrics are additionally pushed to the Weights & Biases (W&B) dashboard [Bie20], enabling real-time monitoring and interactive visualization across runs, client configurations, and partitioning settings.
Compatibility with FedAvg	The aggregation logic of FedAvg remains unaltered, ensuring theoretical consistency with standard FL literature while extending usability.

This strategy design allowed us to keep the computational behaviour of FedAvg intact while introducing an effective experiment management workflow, making it easier to verify, reproduce, and compare results across different numbers of clients, Dirichlet α values, and participation fractions.

3.2 Dataset and Partitioning Strategies

3.2.1 IID and Non-IID Partitioning (Dirichlet)

To study the impact of data heterogeneity in federated learning,the Fashion-MNIST dataset using a Dirichlet distribution-based strategy was partitioned, a common approach for simulating realistic non-IID client dataset, as discussed in [HQB19, LKSJ20]. In Dirichlet partitioning, the label distribution for each client is sampled from a Dirichlet distribution with concentration parameter α, which controls the degree of heterogeneity:

$$\mathbf{p}_k \sim \text{Dirichlet}(\alpha \cdot \mathbf{1}),$$

where $\mathbf{p}_k$ is the class-distribution vector for client k.

Interpretation of α

- $\alpha \to \infty$: all clients receive a nearly identical class distribution (IID).
- $\alpha = 1$: moderate heterogeneity, distributions vary but remain balanced.
- $\alpha \ll 1$: clients receive highly skewed, label-dominated datasets.

During the implementation, the Dirichlet sampler was applied via the DirichletPartitioner from the Flower Datasets API, configured as follows:

```
partitioner = DirichletPartitioner(
    num_partitions=num_clients,
    partition_by="label",
    alpha=<value>,
    seed=42,
)
```

Each client then splits its assigned partition into an 80% local training set and a 20% local test set. Data loaders apply standard Fashion-MNIST transformations (rescaling, normalization), ensuring consistent preprocessing across all clients.

To illustrate the effects of different Dirichlet α values,the number of training samples assigned to each client for four α settings was computed. Table 3.3 summarizes these results.

Table 3.3: Client dataset sizes under different Dirichlet α values (10 clients). Lower α yields higher imbalance.

α	**C1**	**C2**	**C3**	**C4**	**C5**	**C6**	**C7**	**C8**	**C9**	**C10**
10.0	5118	5323	5700	5989	6173	6241	6312	6346	6369	6429
1.0	3875	5208	5450	6166	6166	6224	6235	6481	6627	7568
0.1	683	4676	4830	6220	6304	6967	7357	7641	7653	7669
0.01	76	105	224	4987	5709	6261	6601	11799	11867	12371

As shown, $\alpha = 10$ produces nearly uniform client sizes, while $\alpha = 0.01$ induces extreme imbalance, with some clients receiving nearly all samples and others receiving fewer than 100. This tunable heterogeneity allows systematic evaluation of how non-IID conditions degrade convergence and accuracy—an important aspect of real-world IoT deployments, where devices often collect application-specific, skewed data.

Additionally, flower also provides the option to easily visualize the label data distribution of the data for the corresponding partitions. This can be seen in the figures 3.3 and the 3.4, where both extreme opposites of the value of alpha are visualized.

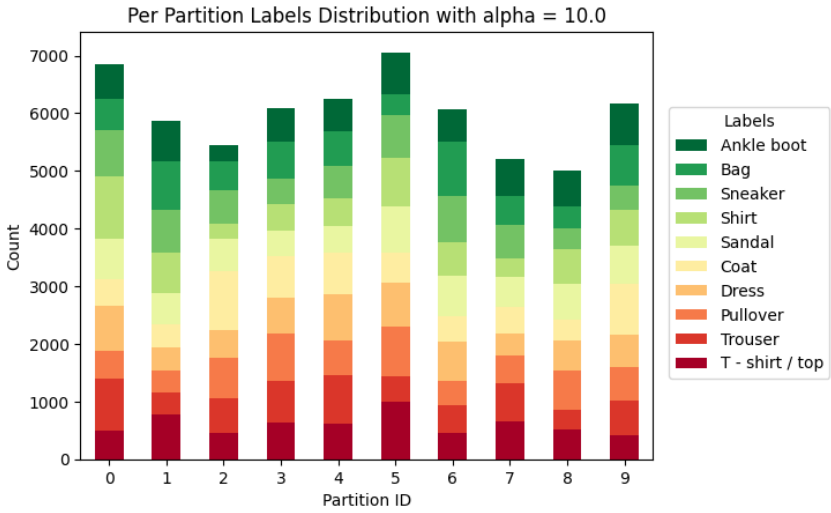


Figure 3.3: Dirichlet Partitioner Alpha 10.0

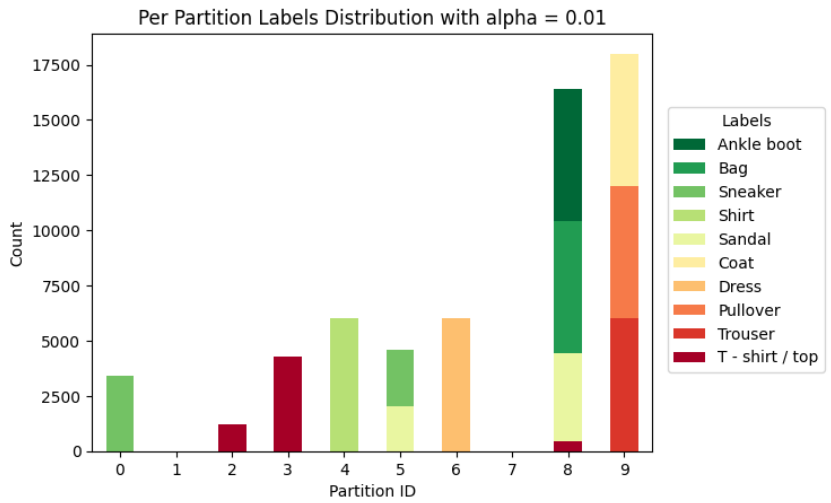


Figure 3.4: Dirichlet Partitioner Alpha 0.01

3.2.2 IID and Non-IID Partitioning (Shard)

Additionally, the flower package also allows the user to utilize different partitioners. Although the Dirichlet partitioner was used to represent IID and non-IID scenarios, the shard partitioner was also used to represent a different scenario of a non-IID case. In this case, it can be assumed that the clients have limited access to specific labels or classes inside the dataset, for example, client 1 can only receive data from "Ankle boot" and "bag", while client 2 can receive "sneaker" and "shirt" and so on. This also represents a real life scenario, as the devices are physically distributed in different places, and therefore have a different access to the real data. A small comparison between partitioners, including Dirichlet and Shard can be seen in the image 3.5. This figure also includes the IidPartitioner, which can also be achieved by the DirichletPartitioner with a really high alpha, which was covered in the previous section of this work.

Figure 3.5: Partitioners

It is important to understand that, whenever the shard partitioner is used, the user cannot control which specific labels are assigned to each specific client. Although, it might be possible with a workaround, the goal was to use the flower library, as it was done in all the experiments of this work. A possible representation of the labels can be seen in the figure 3.6, where it shows a possible random combination of how the clients can get to use the labels of the fashion mnist dataset.

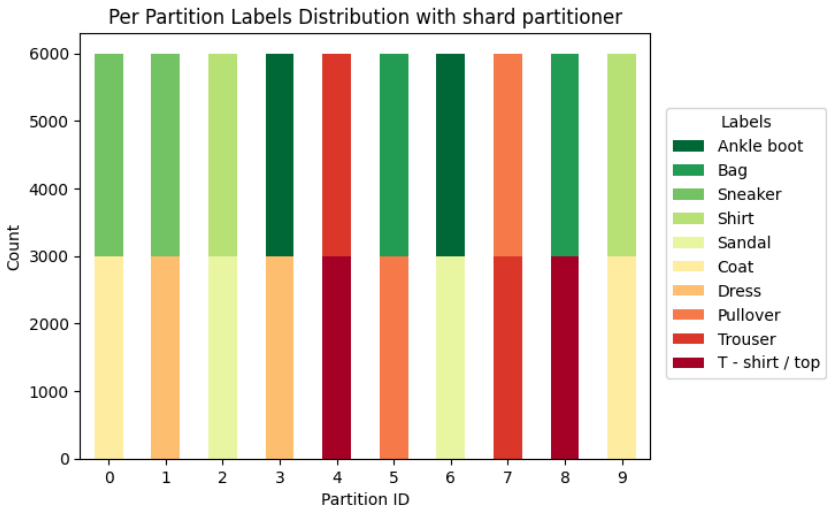


Figure 3.6: Shard partitioner random combination example

3.3 Experiment Configuration

3.3.1 Client Counts and Random Seeds

As mentioned before, this work evaluated how system scale and randomness affect convergence. Experiments were run for multiple client counts (3, 5, 10, 20) and several random seeds to ensure stability across runs, following the recommendations in [WND+22]. The table 3.4 summarizes the experiment configurations.

Table 3.4: Federated learning experiment configurations.

Experiment	Variable	Values
Client scaling	Number of clients	3, 5, 10, 20
Randomness	Random seeds	0–4
Data heterogeneity	Dirichlet α and Shard	0.1, 1.0, 10.0
Client availability	Participation fraction	0.2, 0.5, 1.0

3.3.2 Participation Fraction Experiments

To reflect realistic IoT constraints, we varied the participation fraction (`fraction_fit`). In each round, only a subset of clients performed local training. As observed in prior work [KMA+21], partial participation often yields similar accuracy to full participation when the number of rounds is sufficiently large.

This behaviour was confirmed in our experiments: accuracy and convergence were largely unaffected by reducing participation to 20–50%, demonstrating FL's robustness in device-intermittent IoT environments. However, this will be deeply discussed in the next section of the work.

4

Results and Discussion

This chapter presents the experimental results obtained from varying the number of clients, random seeds, participation fractions, and non-IID Dirichlet parameters, which also includes the Shard partitioner experiment. All experiments were conducted using the FashionMNIST dataset and the CustomFedAvg aggregation strategy described in the previous chapter.

4.1 Effect of Number of Clients

The first set of experiments analyzed how the number of simulated clients influences convergence speed and final accuracy. Client counts ranged from small-scale (e.g., 2 clients) to larger-scale settings (up to 20 clients), with each client receiving a partition of the dataset determined by the Dirichlet mechanism.

Across all settings, the global model exhibited stable convergence. Increasing the number of clients generally reduced the number of local samples per client, resulting in slightly higher variance in early training rounds but no significant degradation in final accuracy. This behavior aligns with observations in prior federated learning work [MMR+17], where the aggregation process compensates for reduced local dataset sizes.

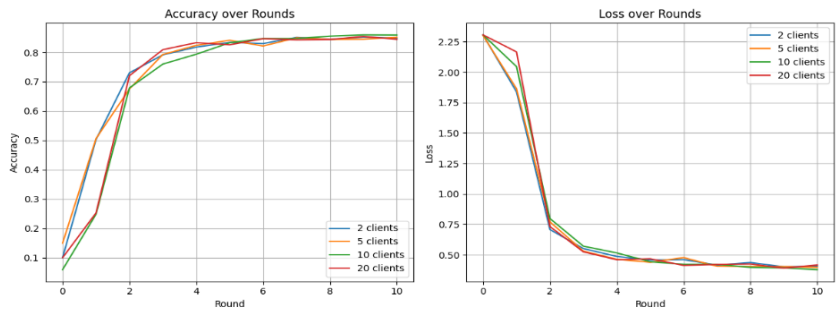


Figure 4.1: Accuracy and loss metrics through rounds

The results, which can be seen in figure 4.1 indicate that federated training remains robust even as the number of clients increases, making the approach suitable for IoT systems where device populations are large and diverse.

4.2 Impact of Random Seeds

To evaluate the stability of the training procedure, multiple runs were performed using different random seeds. These seeds affected data partitioning, weight initialization, and batching order. The resulting accuracy distributions remained tightly grouped across all seeds, demonstrating high reproducibility of the federated setup, which can be seen in the boxplots in the 4.2

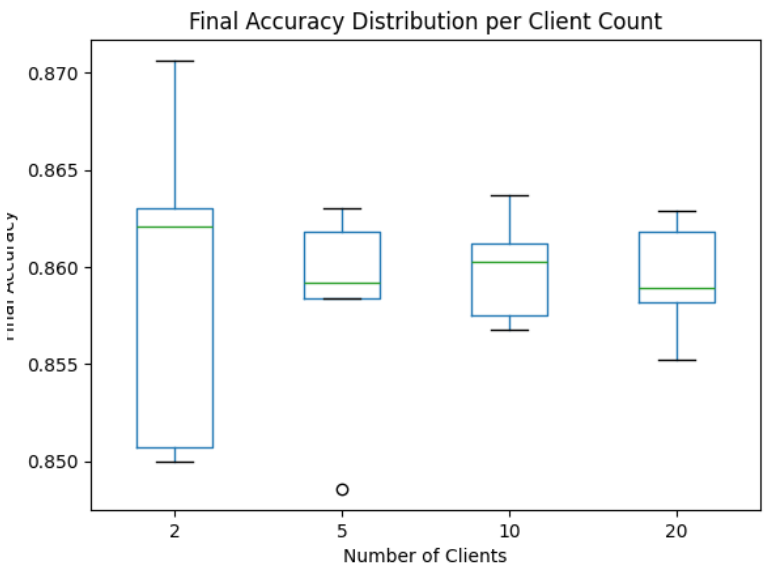


Figure 4.2: Overview of the model accuracy with multiple seeds

The small variance in results suggests that the global model and aggregation strategy are stable even under different stochastic initial conditions. However, the following keypoints can be summmarized in 4.1

Table 4.1: Summarized keypoints.

Characteristic	**Explanation**
Fewer Clients means higher variability	Data per client becomes more random and sensitive to seed initialization.
More Clients means higher stability	Aggregation strategy benefits from more diverse and balanced data
Trade-off	Increasing clients reduces samples per client
Overall effect	Mean accuracy stays roughly constant with more clients

4.3 Influence of Partial Client Participation

Another series of experiments investigated the effect of varying the participation fraction, which controls the proportion of clients performing local training in each round. Fractions ranged from 0.2 (20% participation) to 1.0 (full participation).

The results shown in figure 4.3 demonstrate minimal differences in accuracy or convergence behavior across participation levels. Even with only 20% of clients active per round, the global model maintained performance comparable to full participation conditions.

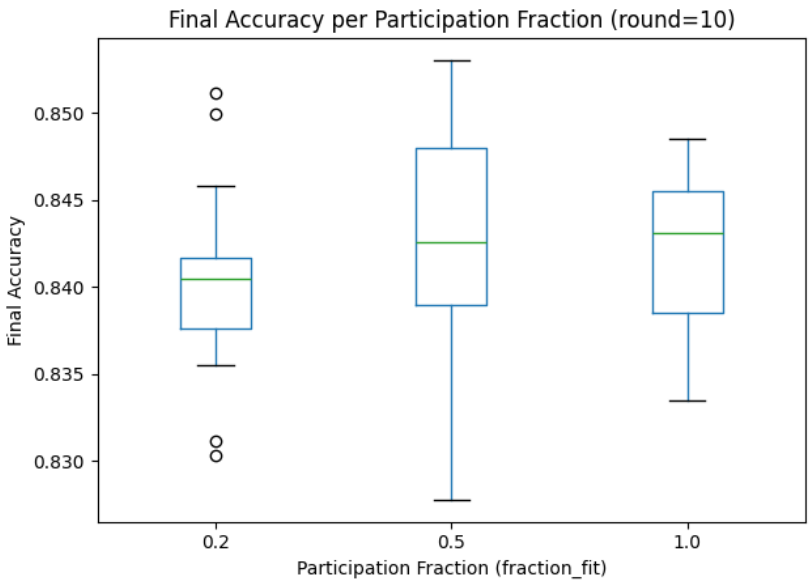


Figure 4.3: Partial Client Participation analysis

This outcome aligns with theoretical expectations: repeated aggregation over multiple rounds ensures that all clients contribute sufficiently over time. This property is particularly relevant for IoT settings where device availability may fluctuate due to connectivity or resource constraints.

4.4 Non-IID Data Effects

4.4.1 Dirichlet non-IID case

The strongest performance differences emerged from modifying the Dirichlet concentration parameter α, which controls the degree of non-IIDness. Results consistently showed that smaller α values (more skewed label distributions) led to; lower final accuracy, slower convergence, and increased variability between clients, as shown in figure 4.4 and figure 4.5

These effects are expected because highly non-IID distributions cause each client to train on narrow or biased subsets of the data, limiting the generalization capability of local updates [HQB19].

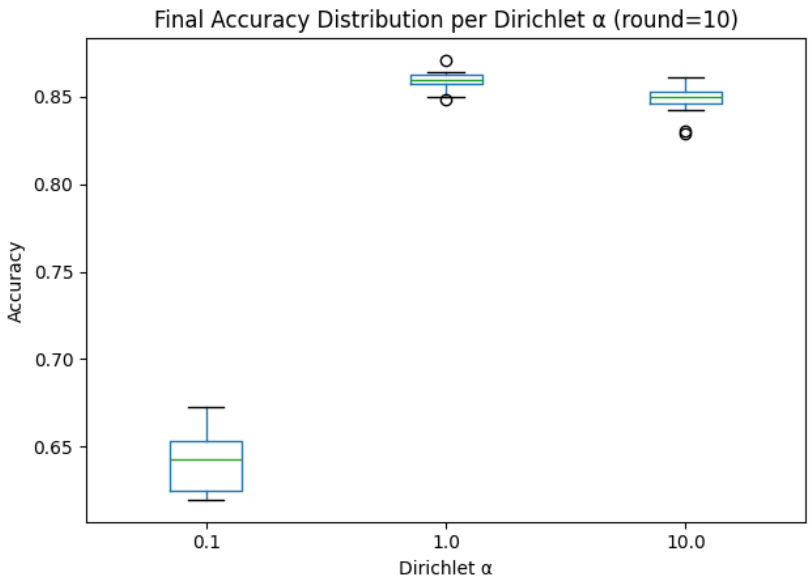


Figure 4.4: Overview of different data distributions

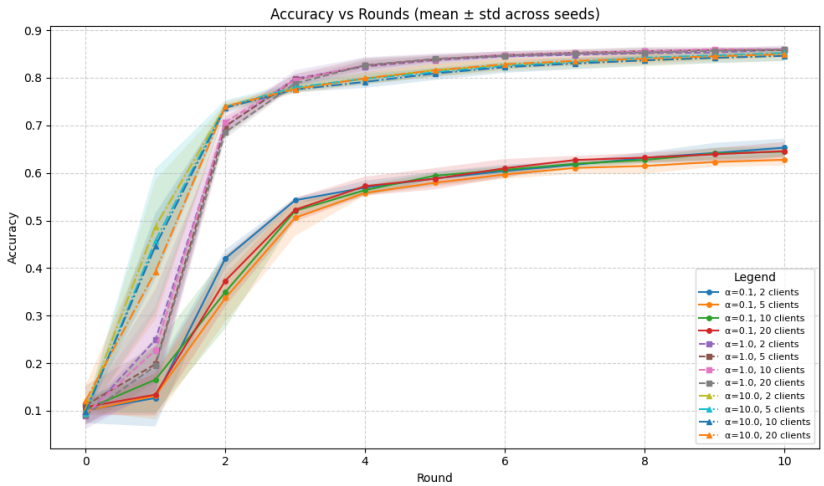


Figure 4.5: Influence of client distribution throughout rounds

The observed degradation at very low α mirrors real-world IoT deployments, where devices often collect highly localized data and produce non-uniform label distributions.

4.4.2 Shard Partitioner non-IID case

The behavior of the previous subsection is way more notorious when the shard partitioner was implemented, as it also belongs to a non-IID scenario. However, in this cases, limiting the clients to only possess two labels or classes of the dataset affected way more the overall performance, as it can be seen in the figure 4.6 and 4.7. This behavior can be presented in a real life IoT enviornment, in such cases where the devices are receiving different data within the same IoT scenario. For example, in smart cameras, where camera A sees only the parking lot vehicles, camera B sees bycicles, and camera C sees people walking by. This is called Client Drifting, and it represents one of the current challenges in Federated Machine Learning.

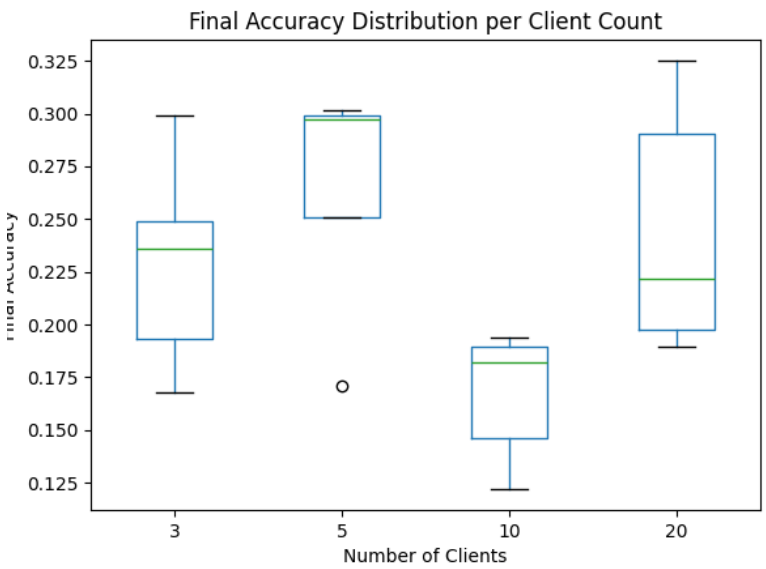


Figure 4.6: Boxplot view of the overall accuracy with shard distribution

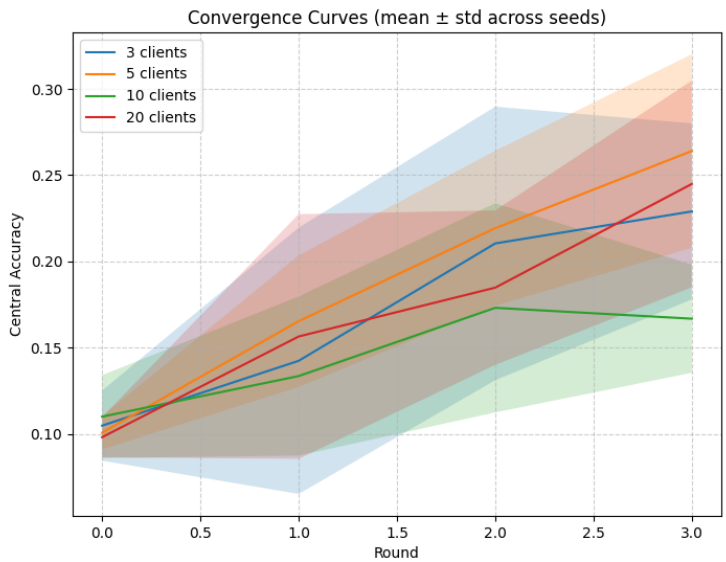


Figure 4.7: Convergence error lines with shard distributions

4.5 Main findings in this work

The experimental findings can be summarized as follows:

- Increasing the number of clients does not significantly harm convergence or final accuracy.
- Training remains stable across different random seeds, indicating strong reproducibility.
- Low participation fractions have minimal negative effect, highlighting suitability for unreliable IoT environments.
- Non-IID partitions (low Dirichlet α or Shard) represent the main challenge, substantially reducing accuracy and convergence speed.

Overall, the results show that Federated Learning using CustomFedAvg is robust to variations in client availability and randomness, but sensitive to data heterogeneity. These tendencies reinforce known limitations of classical FedAvg and underline the importance of designing FL strategies that explicitly address statistical heterogeneity.

5

Conclusion

This chapter summarizes the main outcomes of the federated learning experiments, highlights their implications for real-world IoT deployments, and outlines the limitations that should be considered when interpreting the results.

5.1 Summary of Findings

The experimental evaluation showed that federated learning using the CustomFedAvg strategy can be trained reliably under varying system configurations. The number of clients had only a marginal impact on the final model performance. Although higher client counts introduced additional variance in early rounds due to smaller local datasets, the global model consistently converged as aggregation progressed. The training process also proved to be stable with respect to random seed variation, as multiple independent runs yielded accuracy values concentrated within a narrow range.

The participation fraction was another factor that exhibited limited influence on training dynamics. Even when only a small proportion of clients contributed updates in each round, the global model remained competitive with the full-participation baseline. The most significant source of performance variation arose from the degree of non-IIDness in the data distribution. Experiments with lower Dirichlet parameters or using Shard partitioner resulted in markedly lower accuracy and slower convergence, confirming that statistical heterogeneity remains a fundamental challenge in federated learning. In addition, the CustomFedAvg strategy successfully enabled global checkpointing and automatic metric logging, improving reproducibility and monitoring throughout the experiments.

5.2 Implications for IoT and Real-World Applications

The results carry several implications for federated learning in IoT environments. The robustness to partial participation suggests that federated learning remains functional even when devices are intermittently available due to connectivity fluctuations, energy constraints, or operational schedules. The scalability of the method, reflected by stable performance across different client counts, indicates potential applicability to large-scale IoT systems such as sensor networks or distributed industrial monitoring infrastructures.

However, the sensitivity to highly skewed data distributions highlights a limitation that is particularly relevant in IoT settings. Many edge devices gather highly localized or domain-specific data, leading to distributions that resemble the low-α Dirichlet scenarios observed in the experiments. Performance degradation under such conditions emphasizes the need for federated learning algorithms that explicitly account for statistical heterogeneity. Despite this challenge, the use of a simple convolutional neural network and the Fashion-MNIST dataset demonstrated that federated learning remains computationally feasible on devices with limited resources, further supporting its suitability for practical IoT deployments.

5.3 Limitations

The findings are subject to several limitations related to the experimental setup. The entire evaluation was conducted in a simulated environment rather than on physical IoT hardware. Real devices may introduce additional constraints, including network latency, hardware failures, or limited energy reserves, which were not represented in the simulation. All clients employed the same model architecture, whereas real-world IoT systems often consist of heterogeneous devices with varying computational capacities. This uniformity limits the generalizability of the conclusions.

Furthermore, only a single dataset was used to represent IoT workloads. Fashion-MNIST, while useful for controlled benchmarking, does not capture the multimodal, noisy, or domain-specific data typically encountered in practical IoT scenarios. The study also concentrated on a single strategy, namely FedAvg and its custom variant. More advanced algorithms designed to handle data heterogeneity or device variability could potentially mitigate the performance degradation observed under strongly non-IID conditions. Finally, hyperparameters such as learning rates, optimizers, and model complexity were held fixed throughout the experiments, leaving potential interactions between training configurations and federated behavior unexplored.

Overall, addressing these limitations would provide a more complete understanding of federated learning under realistic IoT constraints and further strengthen the practical relevance of the results.

6

Outlook

6.1 Future Work

Several directions emerge from the results obtained in this study and motivate further investigation. A natural extension involves conducting experiments on real IoT hardware rather than solely relying on simulations. Deploying the training pipeline on heterogeneous embedded devices would make it possible to quantify the effects of computational limitations, intermittent connectivity, and energy constraints on federated performance. Such evaluations would also reveal the practical feasibility of executing local training, exchanging model updates, and managing communication overhead under realistic conditions.

Another important avenue concerns exploring model architectures tailored for resource-constrained environments. Lightweight convolutional networks, quantized models, or architectures specifically designed for microcontrollers could offer more accurate insights into how federated learning behaves in IoT settings. Additional research could also focus on a broader set of datasets that mirror the complexity of real-world multimodal sensor inputs, including time-series data, environmental readings, or audio signals. Investigating optimization techniques, adaptive learning rates, or personalized federated learning methods may further improve convergence stability under varying degrees of statistical heterogeneity.

6.2 Potential Extensions

Several methodological extensions could significantly enhance the robustness, scalability, and privacy guarantees of the system. Integrating differential privacy into the training pipeline would enable formal protections against

information leakage from shared model updates. This addition would be particularly relevant for IoT applications that process sensitive data, such as health monitoring or industrial quality control. However, differential privacy also introduces additional noise, making it necessary to evaluate the trade-off between privacy guarantees and model accuracy.

Another promising extension involves the use of communication-efficient techniques, such as sparsification, quantization, or update compression. These approaches reduce the size of transmitted model parameters and can substantially decrease communication costs, which remain a dominant bottleneck in large-scale federated systems. Incorporating such methods into the existing CustomFedAvg strategy could offer a practical balance between efficiency and convergence.

Beyond model training, federated learning outputs may also be incorporated into larger AI-driven pipelines. One example is the integration of retrieval-augmented generation (RAG) systems, where model predictions or extracted features serve as inputs to downstream decision-support mechanisms. This type of hybrid architecture could enable IoT devices to participate in knowledge-intensive tasks without centralizing raw data. Exploring such combinations would broaden the applicability of federated learning beyond classification and open new possibilities for distributed intelligence across device networks.

Overall, these extensions illustrate the potential for advancing both the methodological and practical aspects of the current system. Continued work in this direction would contribute to a more comprehensive understanding of how federated learning can be optimized for complex, large-scale, and privacy-sensitive IoT deployments.

Bibliography

[ACMF21] Hasanen Alyasiri, John Clark, Ali Malik, and Ruairí de Fréin: *Grammatical Evolution for Detecting Cyberattacks in Internet of Things Environments*. In 2021. DOI: 10.1109/ICCCN52240.2021.9522283.

[ANT17] ANT: *The ANT Lab: Analysis of Network Traffic*. Accessed: 2022-05-16. 2017. URL: https://ant.isi.edu/.

[AIM10] Luigi Atzori, Antonio Iera, and Giacomo Morabito: *The internet of things: A survey*. In *Computer Networks*, volume 54 (15), 2010, pages 2787–2805.

[BTM+20] Daniel J Beutel, Taner Topal, Abhishek Mathur, Xiang Qiu, Titouan Parcollet, and Nicholas D Lane: *Flower: A friendly federated learning framework*. In *Proceedings of the 13th ACM International Conference on Systems for Energy-Efficient Computing and Engineering (e-Energy)*, 2020, pages 33–35.

[Bie20] Lukas Biewald: *Experiment Tracking with Weights and Biases*. https://wandb.com. Software available from wandb.com. 2020.

[BEG+19] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, et al.: *Towards federated learning at scale: System design*. In *Proceedings of Machine Learning and Systems*, volume 1, 2019, pages 374–388.

[Bra23] Alex Braungardt: *Flower, FATE, PySyft & Co.—Federated Learning Frameworks in Python*. Accessed: 2025-11-12. 2023. URL: https://medium.com/elca-it/flower-pysyft-co-federated-learning-frameworks-in-python-b1a8eda68b0d.

[CSG+22] E. M. Campos, P. F. Saura, A. González-Vidal, J. L. Hernández-Ramos, J. B. Bernabé, G. Baldini, and A. Skarmeta: *Evaluating Federated Learning for intrusion detection in Internet of Things: Review and challenges*. In *Computer Networks*, volume 203, 2022, pages 108661.

[CH22] S. Chatterjee and M. K. Hanawal: *Federated Learning for Intrusion Detection in IoT Security: A Hybrid Ensemble Approach*. In *International Journal of Internet of Things and Cyber-Assurance*, volume 2, 2022, pages 62–86.

[CLL+20] Z. Chen, N. Lv, P. Liu, Y. Fang, K. Chen, and W. Pan: *Intrusion Detection for Wireless Edge Networks Based on Federated Learning*. In *IEEE Access*, volume 8, 2020, pages 217463–217472.

[AMA+20] Mohammed Ali Al-Garadi, Amr Mohamed, Abdulla Khalid Al-Ali, Xiaojiang Du, Ihsan Ali, and Mohsen Guizani: *A Survey of Machine and Deep Learning Methods for Internet of Things (IoT) Security*. In *IEEE Communications Surveys & Tutorials*, volume 22 (3), 2020, pages 1646–1685. DOI: `10.1109/COMST.2020.2988293`. URL: `https://fardapaper.ir/mohavaha/uploads/2020/11/Fardapaper-A-Survey-of-Machine-and-Deep-Learning-Methods-for-Internet-of-Things-IoT-Security.pdf`.

[GBMP13] Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami: *Internet of Things (IoT): A vision, architectural elements, and future directions*. In *Future Generation Computer Systems*, volume 29 (7), 2013, pages 1645–1660.

[HQB19] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown: *Measuring the Effects of Non-Identical Data Distribution for Federated Learning*. In *NeurIPS Workshop on Federated Learning*, 2019.

[KMA+21] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, et al.: *Advances and open problems in federated learning*. In *Foundations and Trends® in Machine Learning*, volume 14 (1–2), 2021, pages 1–210.

[LTC23] Riccardo Lazzarini, Huaglory Tianfield, and Vassilis Charissis: *Federated Learning for IoT Intrusion Detection*. In *AI*, volume 4 (3), 2023, pages 509–530. DOI: `10.3390/ai4030028`. URL: `https://doi.org/10.3390/ai4030028`.

[LSTS20] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith: *Federated learning: Challenges, methods, and future directions*. In *IEEE Signal Processing Magazine*, volume 37 (3), 2020, pages 50–60.

[LZ18] Yang Li and Jia Zhan: *Learning IoT in edge: Deep learning for the Internet of Things with edge computing*. In *IEEE Network*, volume 32 (1), 2018, pages 96–101.

[LNJK+20] Wei Yang Bryan Lim, Nguyen Cong Nguyen, Jihong Jin, Dinh C Kim, et al.: *Federated learning in mobile edge networks: A comprehensive survey*. In *IEEE Communications Surveys & Tutorials*, volume 22 (3), 2020, pages 2031–2063.

[LKSJ20] Tao Lin, Lingjing Kong, Sebastian U Stich, and Martin Jaggi: *Ensemble Distillation for Robust Model Fusion in Federated Learning*. In *NeurIPS Workshop on Scalability, Privacy, and Security in Federated Learning*, 2020.

[MMR+17] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas: *Communication-efficient learning of deep networks from decentralized data*. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017, pages 1273–1282.

[MKP+22] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh, A. Dehghantanha, and G. Srivastava: *Federated-Learning-Based Anomaly Detection for IoT Security Attacks*. In *IEEE Internet of Things Journal*, volume 9, 2022, pages 2545–2554.

[MMG21] Mainak Mukherjee, Sudip Misra, and Samiran Ghosh: *Machine learning for the Internet of Things: A survey*. In *IEEE Internet of Things Journal*, volume 8 (10), 2021, pages 7992–8016.

[RHL+20] Nicola Rieke, Jonny Hancox, Wenqi Li, et al.: *The future of digital health with federated learning*. In *npj Digital Medicine*, volume 3 (1), 2020, pages 1–7.

[SMD21] R. Saha, S. Misra, and P. K. Deb: *FogFL: Fog-Assisted Federated Learning for Resource-Constrained IoT Devices*. In *IEEE Internet of Things Journal*, volume 8, 2021, pages 8456–8463.

[SLMP23] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann: *A Cyber Threat Intelligence Sharing Scheme based on Federated Learning for Network Intrusion Detection*. In *Journal of Network and Systems Management*, volume 31 (3), 2023.

[SWMS20] Felix Sattler, Simon Wiedemann, Klaus-Robert Müller, and Wojciech Samek: *Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints*. In *IEEE Transactions on Neural Networks and Learning Systems*, volume 32 (8), 2020, pages 3710–3722.

[SCZ+16] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu: *Edge computing: Vision and challenges*. In *IEEE Internet of Things Journal*, volume 3 (5), 2016, pages 637–646.

[WND+22] Kok-Seng Wong, Manh Nguyen-Duc, Cuong Do-Danh, Khiem Le-Huy, and Danh Le-Phuoc: *An Empirical Study of Federated Learning on IoT-Edge Devices: Resource Allocation and Heterogeneity*. In *IEEE Internet of Things Journal*, 2022.

[WNL+23] Kok-Seng Wong, Manh Nguyen-Duc, Khiem Le-Huy, Long Ho-Tuan, Cuong Do-Danh, and Danh Le-Phuoc: *An Empirical Study of Federated Learning on IoT-Edge Devices: Resource Allocation and Heterogeneity*. In *CoRR*, volume abs/2305.19831, 2023. preprint (arXiv), Accessed: 2025-11-12. URL: `https://arxiv.org/abs/2305.19831`.

[XRV17] Han Xiao, Kashif Rasul, and Roland Vollgraf: *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. In *arXiv preprint arXiv:1708.07747*, 2017.

[YLCT19] Qiang Yang, Yang Liu, Tianjian Chen, and Yongxin Tong: *Federated machine learning: Concept and applications*. In *ACM Transactions on Intelligent Systems and Technology*, volume 10 (2), 2019, pages 1–19.

[ZBC+14] Andrea Zanella, Nicola Bui, Angelo Castellani, Lorenzo Vangelista, and Michele Zorzi: *Internet of Things for smart cities*. In *IEEE Internet of Things Journal*, volume 1 (1), 2014, pages 22–32.

[ZWCW21] Jialin Zhang, Yong Wang, Jialin Chen, and Jun Wu: *A survey on machine learning in Internet of Things: Algorithms, strategies, and applications*. In *IEEE Internet of Things Journal*, volume 8 (11), 2021, pages 8697–8719.

[ZHM+21] T. Zhang, C. He, T. Ma, L. Gao, M. Ma, and S. Avestimehr: *Federated Learning for Internet of Things: A Federated Learning Framework for On-device Anomaly Data Detection*. In *arXiv preprint*, volume arXiv:2106.07976, 2021.

[ZYZM23] Yongqiang Zhang, Hongchang Yu, Wanzhen Zhou, and Menghua Man: *Application and Research of IoT Architecture for End-Net-Cloud Edge Computing*. In *Electronics*, volume 12 (1), 2023. ISSN: 2079-9292. DOI: `10.3390/electronics12010001`. URL: `https://www.mdpi.com/2079-9292/12/1/1`.

[ZLLL22] L. Zhao, J. Li, Q. Li, and F. Li: *A Federated Learning Framework for Detecting False Data Injection Attacks in Solar Farms*. In *IEEE Transactions on Power Electronics*, volume 37, 2022, pages 2496–2501.

[ZYSX20] R. Zhao, Y. Yin, Y. Shi, and Z. Xue: *Intelligent intrusion detection based on federated learning aided long short-term memory*. In *Physics and Communication*, volume 42, 2020, pages 101157.

[ZCL+19] Zhi Zhou, Xu Chen, En Li, Li Zeng, Kaoru Luo, and Junshan Zhang: *Edge intelligence: Paving the last mile of artificial intelligence with edge computing*. In *Proceedings of the IEEE*, volume 107 (8), 2019, pages 1738–1762.

Eidesstattliche Erklärung

Hiermit versichere ich an Eides statt, dass die vorliegende Arbeit selbstständig verfasst wurde und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt wurden.

Hamburg, am December 8, 2025 Marco Antonio Barrera Salas

Appendix

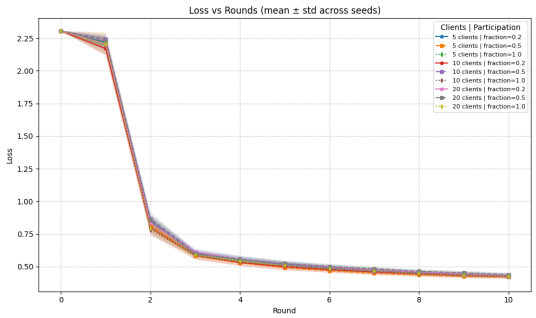


Figure 1: The loss behavior for different client participation

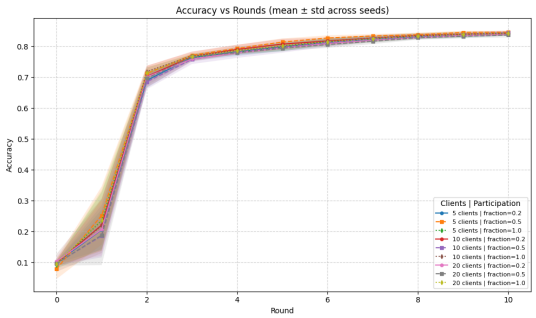


Figure 2: The accuracy behavior for different client participation

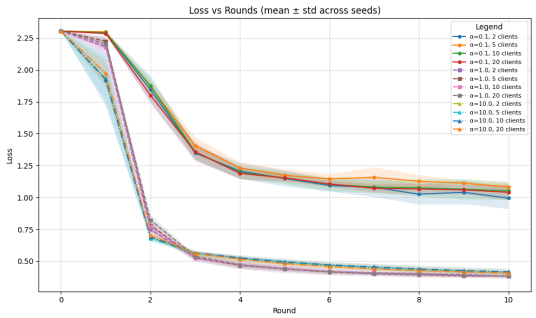


Figure 3: The loss behavior for different data distributions

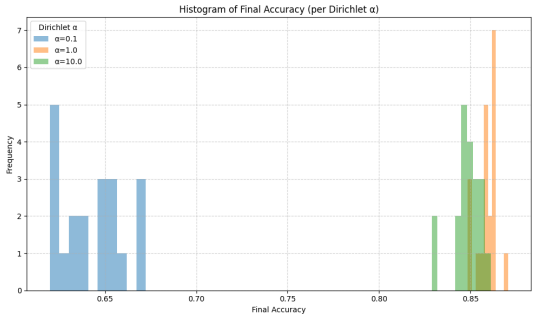


Figure 4: Histogram view for different data distributions

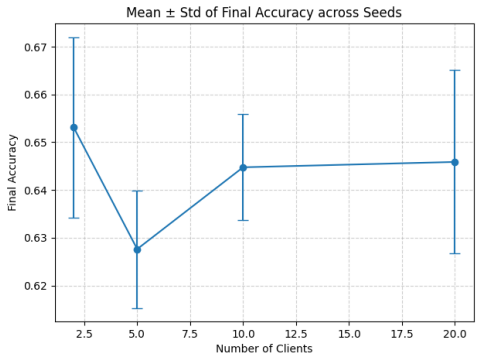


Figure 5: Data behavior for factor 0.1 in the partitioner

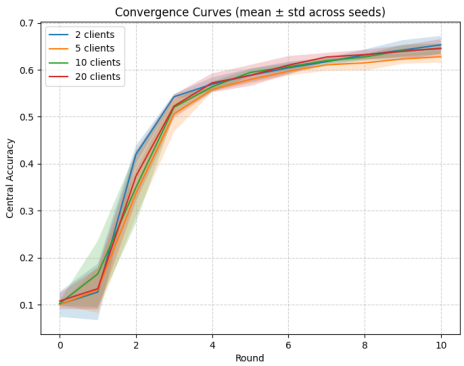


Figure 6: Data behavior for factor 0.1 in the partitioner through rounds

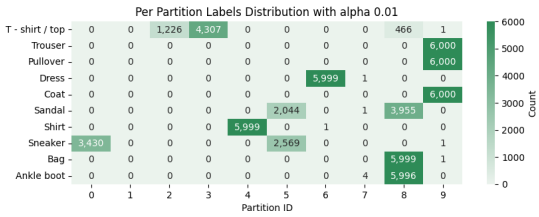


Figure 7: Heatmap for the data distribution with Dirichlet partitioner of alpha 0.01

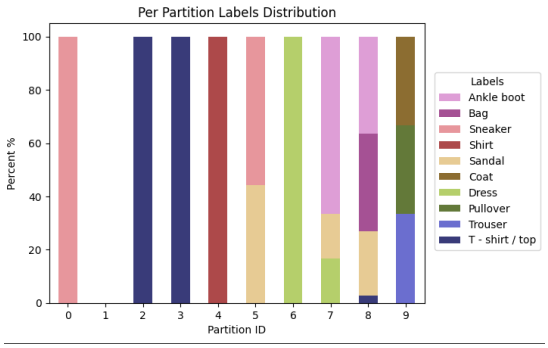


Figure 8: Percentage barplot for the data distribution with Dirichlet partitioner of alpha 0.01

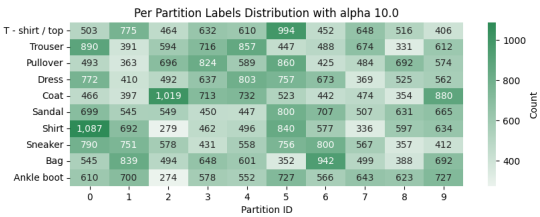


Figure 9: Heatmap for the data distribution with Dirichlet partitioner of alpha 10.0

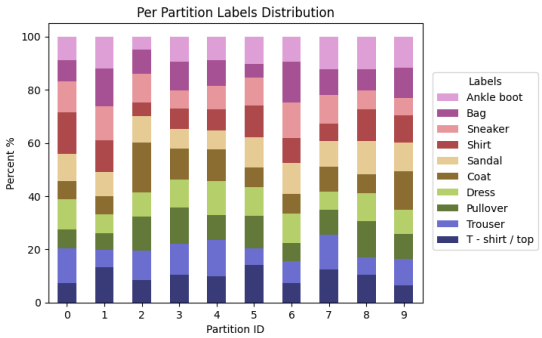


Figure 10: Percentage barplot for the data distribution with Dirichlet partitioner of alpha 10.0